

HW#5: Domain-Specific Accelerator

Yen-Chieh Huang, 109550059

ABSTRACT

This report presents the design and implementation of a Domain-Specific Accelerator (DSA) for accelerating the execution of a MLP model for handwritten digits recognition. The DSA employs pipeline parallelism and data parallelism to optimize computation speed, resulting in a significant speedup (309x) over the software-only implementation. The report provides a detailed overview of the DSA design, including the architecture, data transfer mechanisms, and performance evaluations.

1 AN OVERVIEW OF THE DESIGN

Algorithm 1: An Overview on Interfacing with the DSA

```
// 1. Read data from SD card
Weights, Bias, Images, Labels = ReadDataFromSdCard()
// 2. Write weights & bias to DSA
WriteParametersToDSA(Weights, Bias)
// 3. Write images to DSA
for Index = 0 to NumberOfImages-1 do
    WriteImageToDSA/Images[Index])
end
// 4. Read result from DSA & check correctness
NumCorrect = 0
for Index = 0 to NumberOfImages-1 do
    Result = ReadResultFromDSA()
    if Result == Labels[Index] then
        NumCorrect = NumCorrect + 1
    end
end
end
```

Algorithm 1 provides a high-level overview on interfacing with the DSA. Instead of accelerating solely on inner products, I accelerate the entire MLP model. This approach is motivated by the observation that executing the control flow of MLP computation in software can be time-consuming, and hardwiring these logic in hardware can significantly speed up the overall process. Specifically, focusing solely on accelerating inner products yields an execution time of approximately 2000ms. However, when the entire MLP computation is accelerated, an speedup of around 309x (compared to software implementation) is achieved, resulting in 71ms execution time.

The mechanism for interfacing with the DSA involves a sequence of steps. Initially, weights and biases are written to the DSA, followed by the transmission of all images. Finally, the prediction results of all images are read from the DSA. Notably, my implementation does not necessitate waiting for the completion of MLP computation before sending the next image. This feature enables the system to proceed with the transmission of subsequent images even before the computation for the preceding image is finished. In this

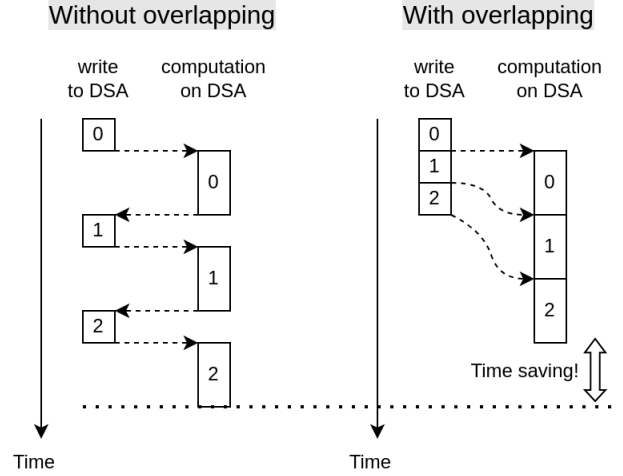


Figure 1: Overlapping data transfer and computation. (The dashed arrow indicates the order must be enforced between data transfer and computation, and the number inside the rectangular box indicates the index of the image.)

way, time spent on data transfer and computation are overlapped, thus accelerating the overall process. The idea of overlapping data transfer and computation is illustrated in Figure 1.

2 TRANSFERRING DATA FROM/TO THE DSA

2.1 The Addressing Space of the DSA

In order to send data to the DSA, I have made some modifications to *soc_top.v*. The modification ensures that when the read/write address falls within the range of 0xC4000000 to 0xC4FFFFFF, signals are directed to the DSA.

It is noteworthy that, for the specified MLP configuration—a fixed 3-layer model with dimensions 784-48-10—the sequence for sending neurons and weights to the DSA has been predetermined during the DSA’s design phase (will be elaborated in subsection 2.2). Consequently, the exact address used to communicate with the DSA (i.e., the XXXXXX in 0xC4XXXXXX) becomes irrelevant, simplifying the data transmission process.

In my implementation, only the address 0xC4000000 is utilized, and all data transmissions occur at this specific location. This deliberate simplification of the address facilitates a more straightforward data exchange and eliminates the need for managing and differentiating between multiple addresses within the address space for the DSA.

2.2 The Sequence of Transferring Data

In Section 1, the process of sending weights, bias, and images to the DSA is abstracted to these lines of code:

Algorithm 2: An Excerpt from Algorithm 1

```
// 2. Write weights & bias to DSA
WriteParametersToDSA(Weights, Bias)
// 3. Write images to DSA
for Index = 0 to NumberOfImages-1 do
    WriteImageToDSA(Images[Index])
end
```

In this subsection, I'll go through the detail of these processes. Below is the pseudocode of WriteParametersToDSA():

Algorithm 3: WriteParametersToDSA()

```
function WriteParametersToDSA(Weights, Bias)
    // A. Write bias of second layer to DSA
    for Index = 0 to NumNeuronsInSecondLayer-1 do
        *(0xC4000000) = BiasOfSecondLayer[Index]
    end
    // B. Write bias of third layer to DSA
    for Index = 0 to NumNeuronsInThirdLayer-1 do
        *(0xC4000000) = BiasOfThirdLayer[Index]
    end
    // C. Send weights between 1st and 2nd layer
    for i = 0 to NumNeuronsInFirstLayer-1 do
        for j = 0 to NumNeuronsInSecondLayer-1 do
            *(0xC4000000) =
                WeightsBetweenFirstAndSecondLayer[i][j]
        end
    end
    // D. Send weights between 2nd and 3rd layer
    for i = 0 to NumNeuronsInSecondLayer-1 do
        for j = 0 to NumNeuronsInThirdLayer-1 do
            *(0xC4000000) =
                WeightsBetweenSecondAndThirdLayer[i][j]
        end
    end
```

And below is the pseudocode of WriteImageToDSA():

Algorithm 4: WriteImageToDSA()

```
function WriteImageToDSA(Image)
    for Index = 0 to NumImagePixels-1 do
        *(0xC4000000) = Image[Index]
    end
```

It is noteworthy that the weights and biases are transmitted in a predetermined order, and adherence to this specific order is necessary when writing an application targeted for my DSA because the internal logic of the DSA is hardwired to expect data in this specific sequence. A further explanation of the implementation, especially regarding how these parameters are stored in the DSA, will be elaborated in Section 3.

3 DESIGN OF THE DSA

The designed DSA contains multiple components as shown in the block diagram of Figure. 2. For the sake of clarity, complex control signals have been deliberately omitted to focus on the datapath. The DSA contains four components: (A) *FIFOs*, (B) *controller*, (C) *multiply-add & ReLU datapath*, and (D) *result datapath*. The functions of each component are described as follows:

- (A) The *FIFOs* act as buffers that facilitate seamless communication between external and internal system components. When the computing unit is busy and cannot receive new data, overlapping computations and data transfer can be achieved by temporarily storing data in the FIFO.
- (B) The *controller* arbitrates the flow of data and maintains state machines that track the progress of MLP computation.
- (C) The *multiply-add & ReLU datapath* is responsible for fetching input images and parameters from FIFOs or registers, performs multiplication, addition, or ReLU operations and subsequently writes the results back to registers that store neurons.
- (D) The *result datapath* iterates through all neurons in the third layer, keeping track of both the value and the index of the largest neuron. It then writes the result (predicted label) into the output FIFO.

It is important to note that the DSA incorporates multiple multiply-add & ReLU datapaths and result datapaths to achieve data parallelism. This design allows for the simultaneous processing of multiple images, enhancing the accelerator's overall computational efficiency. Further elaboration on data parallelism will be provided in subsection 3.6.

The following subsections will delve into a more detailed examination of each individual component, describing their roles and functions within the DSA.

3.1 FIFOs

In the DSA, three FIFOs—the *image FIFO*, *parameter FIFO*, and *result FIFO*—are utilized, each implemented through BRAM. Designed akin to a ping-pong buffer, these FIFOs facilitate the overlap of data transfer and computation, as demonstrated in Figure 1. The *image FIFO* stores the input image, the *parameter FIFO* stores the weights and bias of the MLP model, and the *result FIFO* stores the predicted label.

Initially, the input data to the DSA is directed into the parameter FIFO. Once all weights and biases are received, the input data is redirected to the image FIFO, and concurrently, the output of the parameter FIFO is looped back to its own input. This ensures that the parameter FIFO continuously circulates the same sequence of weights and biases, eliminating the need to repeatedly fetch identical parameters from the CPU. This optimized approach is particularly beneficial as weights and biases are reused for each image, contributing to the overall efficiency of the DSA.

3.2 Controller

The *controller* plays a pivotal role in orchestrating the flow of data and maintaining the state of computation within the circuit. In my design, the controller employs two distinct state machines—the *feeder state machine* and the *receiver state machine*. Additionally,

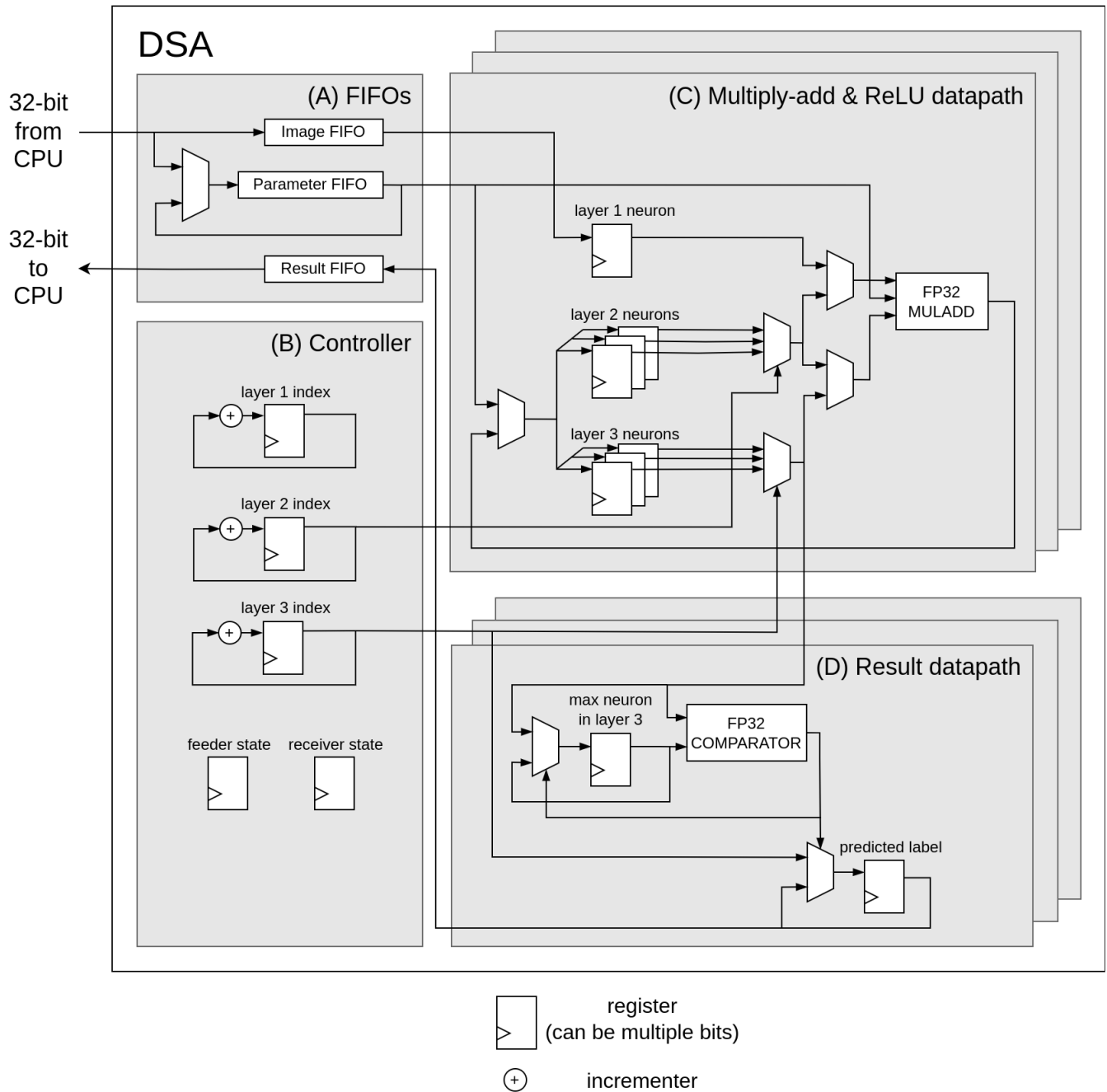


Figure 2: The block diagram of the DSA.

for each layer, one register is utilized to track the index of neuron that is undergoing computation.

- The *feeder state machine* is responsible for monitoring the progress of MLP computation and directing the corresponding data into floating-point units (multiply-adder or comparator). This state machine is crucial in orchestrating the flow of data to floating-point units.
- The *receiver state machine* is responsible for monitoring the progress of MLP computation and directing the output of floating-point units into the appropriate registers. For example, during the computation of the inner product, the output of multiply-adder is directed to registers that store the values of neurons. This state machine ensures that computation results are accurately stored and managed.

These two state machines operate independently for most of the time, facilitating high overlap in data processing—i.e., sending data into floating-point units and receiving data from them simultaneously. However, synchronization between these two state machines becomes necessary at certain points to ensure the correctness of computations. For example, before feeding data into the floating-point comparator to select the maximum neuron in the last layer, the values of neurons in the last layer must be valid. Consequently, it becomes imperative to wait for all computations within the multiply-adder to be completed before initiating the process of feeding data to the comparator.

3.3 Multiply-add & ReLU Datapath

The *multiply-add & ReLU datapath* contains several registers designed to store the neuron values in each layer. In the multiply-add operation ($P=A \times B + C$), the input A is sourced from neurons in either the first or second layer, input B is from the parameter FIFO (which contains weights), and input C is from neurons in either the second or third layer. The resulting output P is then written back to neurons in either the second or third layer. The selection of neurons in each layer is determined by the index registers within the controller.

The ReLU operation is implemented by examining the sign bit of the neuron values. If the sign bit is 1, indicating a negative value, the outcome of the ReLU operation ensures that the value becomes zero.

3.4 Result Datapath

The *result datapath* iterates through all neurons in the third layer and keeps track of the index and value of largest neuron. The comparison is performed by feeding the neuron values into the floating-point comparator. After finding the largest neuron, the result is written into the result FIFO, ready to be retrieved by the CPU.

3.5 Exploiting Pipeline Parallelism

In my design, the latency of floating-point multiply-adder and comparator are 16 and 2 cycles, respectively. While these latencies could be reduced to 1 cycle without incurring timing errors, opting for a higher latency is advantageous for the overall robustness of the design, as it enables the DSA to operate at higher clock frequencies.

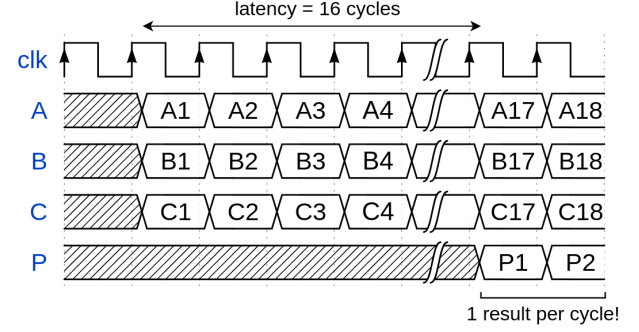


Figure 3: The timing diagram of floating-point multiply-adder. ($P=A \times B + C$ and the latency of computation is 16 cycles.)

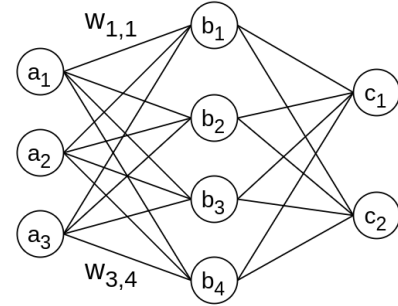


Figure 4: A toy MLP model. (a_i, b_j, c_k are neurons, and $w_{x,y}$ is the weight between a_x and b_y)

However, designing pipeline for floating-point units that has latency greater than 1 can be challenging. This subsection describes the strategies employed to keep the floating-point units busy and achieves an approximate throughput of 1 result per cycle. Without loss of generality, only multiply-adder is discussed in this subsection, but the same idea can be applied to the comparator. The timing diagram in Figure 3 illustrates that, following the latency of 16 cycles, a new result is produced every subsequent cycle, underscoring the potential of pipeline parallelism in enhancing computational throughput.

To apply pipeline parallelism on computing MLP model, a special order of computation need to be enforced to avoid data dependencies (similar to the idea of avoiding data hazard in CPU). To illustrate how to apply pipeline parallelism on computing MLP model, a toy MLP model is introduced to explain the idea (Figure 4). Suppose the latency of multiply-adder is 3 cycles, and before starting the computation, b_i are initialized to be their corresponding biases.

The timing diagram of the multiply-adder after applying pipeline parallelism on the toy model is illustrated in Figure 5. The following paragraph provides a detailed description of the input/output of the multiply-adder for each cycle. Cycle numbers are denoted above the clk signal, presented inside a circle, such as ①, ②, ③:

- Cycle ①: $A = a_1, B = w_{1,1}, C = b_1$.
- Cycle ②: $A = a_1, B = w_{1,2}, C = b_2$.
- Cycle ③: $A = a_1, B = w_{1,3}, C = b_3$.

- Cycle ④: $A = a_1$, $B = w_{1,4}$, $C = b_4$. At this cycle, the result of cycle ① is completed (denoted as $P = b'_1$).
- Cycle ⑤: $A = a_2$, $B = w_{2,1}$, $C = b'_1$. At this cycle, the result of cycle ② is completed (denoted as $P = b'_2$).
- Cycle ⑥: $A = a_2$, $B = w_{2,2}$, $C = b'_2$. At this cycle, the result of cycle ③ is completed (denoted as $P = b'_3$).
- Cycle ⑦: $A = a_2$, $B = w_{2,3}$, $C = b'_3$. At this cycle, the result of cycle ④ is completed (denoted as $P = b'_4$).
- Cycle ⑧: $A = a_2$, $B = w_{2,4}$, $C = b'_4$. At this cycle, the result of cycle ⑤ is completed (denoted as $P = b''_1$).

This approach effectively maximizes the utilization of the multiply-adder, generating one result per cycle. This methodology can be scaled for more extensive models and multiply-adders with larger latency. However, care must be taken when the latency exceeds the number of intermediate neurons. For instance, consider the same toy model as Figure 4, but the multiply-adder's latency is extended to 5 cycles. In this situation, the input C in multiply-adder will be incorrect after cycle ⑤. Figure 6 illustrates the erroneous timing diagram, where at cycle ⑤, input C should be b'_1 (the result from cycle ①). However, b'_1 is not yet valid until cycle ⑥, leading to incorrect results.

In my design, this problem is addressed by introducing idle cycles for the multiply-adder when necessary, as depicted in Figure 7. While this strategy may slightly impact the throughput of the

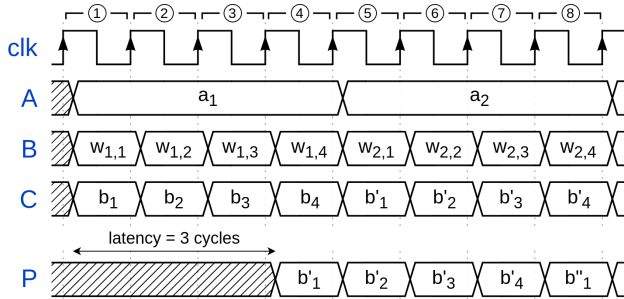


Figure 5: The timing diagram of multiply-adder after applying pipeline parallelism on the toy model. (Assume the latency of computation is 3 cycles.)

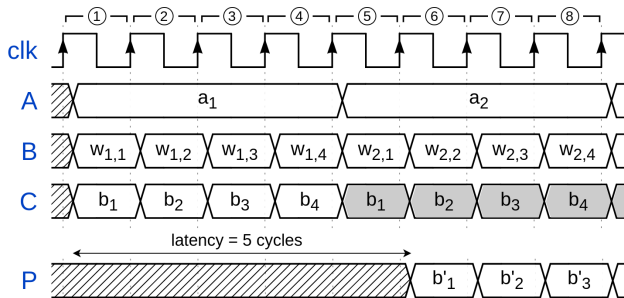


Figure 6: The erroneous timing diagram of multiply-adder. (Assume the latency of computation is 5 cycles. Wrong inputs are highlighted in grey.)

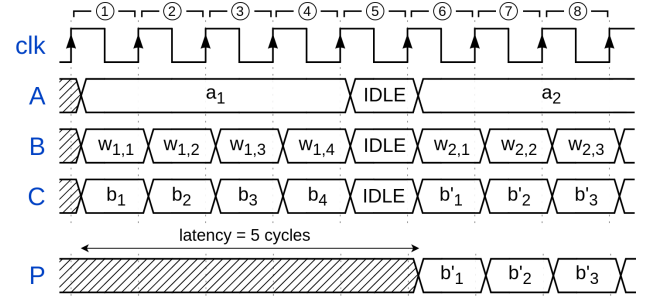


Figure 7: The correct timing diagram of multiply-adder by introducing idle cycles when necessary.

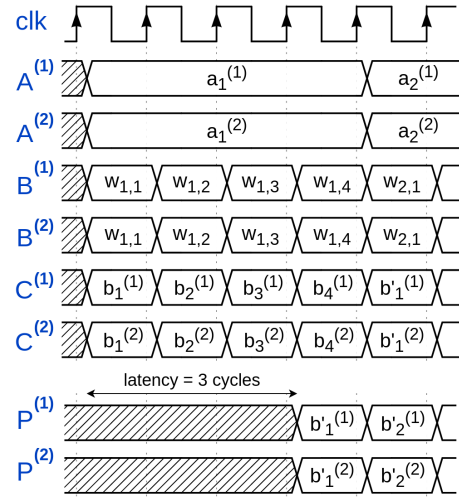


Figure 8: The timing diagram of the multiply-adder after applying pipeline parallelism and data parallelism on the toy model. (The number inside parentheses indicates the index of the image. For example, $a_1^{(2)}$ is the 2nd image's a_1 .)

multiply-adder, the overall impact is negligible as idle cycles are infrequent. Section 4 will further elaborate on the performance enhancements achieved by exploiting pipeline parallelism.

3.6 Exploiting Data Parallelism

Another optimization for speeding up computation is by exploiting data parallelism. Since the computation for different images are independent, we can use multiple floating point units to compute them in parallel. The timing diagram after applying pipeline parallelism and data parallelism is shown in Figure 8.

To implement data parallelism, multiple floating-point units and registers are instantiated, and the read width of the image FIFO is set to $32N$ bits, ensuring the simultaneous processing of N images. The addition of data parallelism has the potential to speed up computation by N times. The performance impact after applying data parallelism will be further discussed in section 4.

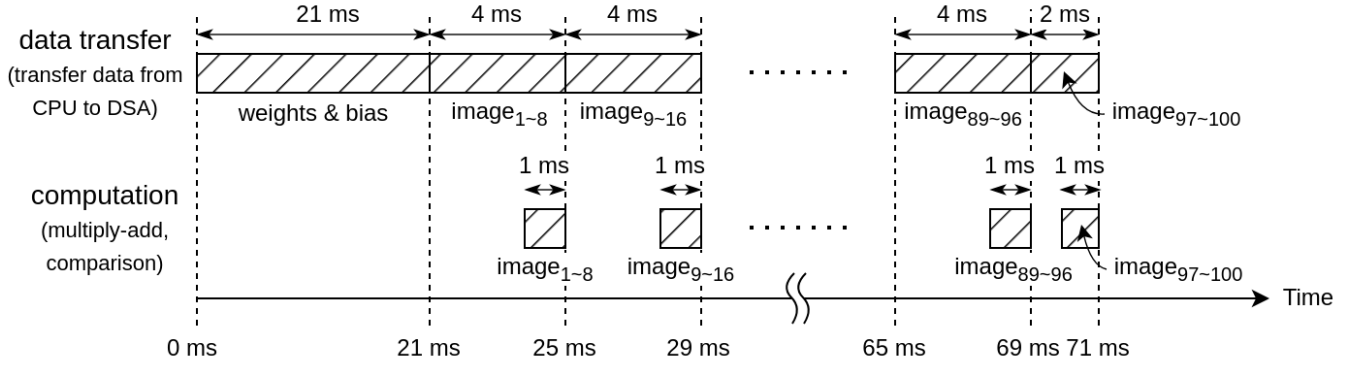


Figure 9: The time diagram of data transfer and computation.

Table 1: Time Spent on Hand-Written Digits Recognition Test. (SW: software-only implementation. PP: hardware-accelerated by pipeline parallelism. PP+DP: hardware-accelerated by pipeline parallelism and data parallelism.)

Implementation	Time (ms)	Speedup
SW	21904	-
PP	116	189x
PP+DP	71	309x

4 PERFORMANCE EVALUATION

Table 1 illustrates the execution time of the hand-written digits recognition test using various implementations, including a software-only version (SW) and hardware-accelerated versions with pipeline parallelism (PP) and a combination of pipeline and data parallelism (PP+DP). The hardware implementations demonstrate significant speedup over the software counterpart, with PP+DP achieving the highest speedup of 309x. The following subsections describe the detailed analysis of each implementation.

4.1 Software-only Implementation (SW)

The SW implementation is the unmodified version of the provided code and is inherently slow due to the absence of floating-point unit (FPU) in CPU. The lack of FPU necessitates emulation of floating-point operations in software, resulting in slower performance. Additionally, the presence of multiple control branches, such as for loops of the MLP computation, makes the software implementation highly reliant on the accuracy of the branch predictor. Nevertheless, the slow performance indicates ample room for improvement through hardware acceleration.

4.2 Hardware-Accelerated by Pipeline Parallelism (PP)

The PP implementation maximizes the utilization of the floating-point units through pipeline parallelism, and only one floating-point multiply-adder and one comparator are utilized in this implementation. The overall execution time is 116 ms, with 21 ms spent on sending parameters from the CPU to the DSA. The assessment

of efficiency is conducted with reference to roofline performance, specifically considering the deployment of only one floating-point multiply-adder and one comparator.

The throughput of both the multiply-add and comparison are limited to 1 per cycle due to the use of a single multiply-adder and comparator. Consequently, the minimum time required for computing one image is approximately $(784 \times 48 + 48 \times 10 + 10) / 41666.67 \approx 0.91$ ms. Hence, the completion of computations for the entire set of images (100 in total) necessitates a minimum duration of 91 ms. In my implementation, $116 - 21 = 95$ ms is spent on MLP computation, indicating proximity to the roofline performance.

4.3 Hardware-Accelerated by Pipeline Parallelism and Data Parallelism (PP+DP)

The PP+DP implementation aims to further enhance performance by instantiating multiple floating-point units and simultaneously processing multiple images. Despite configuring the degree of data parallelism to 8, the observed speedup over the PP implementation is merely 1.63x. The bottleneck is identified through the time diagram of data transfer and computation (Figure 9). Data transfer inefficiency limits overall efficiency, as most of the time, floating-point units are idle, waiting for image data from the CPU.

The root cause of the sluggish data transfer lies in the inherent overhead of the memory subsystem. The absence of DMA in the design necessitates data fetching from CPU, which, in turn, fetches data from the cache, and the cache fetches data from DRAM. This extended path complicates data transfer and reduces efficiency.

To solve this problem, addressing the slow data transfer becomes imperative. Specifically, attention should be directed towards mitigating the inherent inefficiencies in the memory subsystem. Introducing DMA support in the DSA design may be effective in accelerating data transfer, reducing idle times in the floating-point units, and ultimately enhancing the overall efficiency of the PP+DP implementation.

5 CONCLUSION

In this report, a DSA for accelerating the execution of a MLP model for handwritten digits recognition was presented. The DSA employs several types of parallelism, resulting in a substantial speedup

over the software-only implementation. The PP implementation achieved a speedup of 189x, and the PP+DP implementation, which introduced data parallelism, further improved the speedup to 309x. However, the analysis revealed that data transfer inefficiencies, particularly in the memory subsystem, limit the overall performance

gains. To address this bottleneck, future DSA design could incorporate DMA support to streamline data transfer and minimize idle times in the floating-point units. Overall, the presented DSA design demonstrates the potential for significant acceleration in the execution of MLP models, and further optimizations can be explored to maximize its efficiency.